MiniOS Hardware Drivers (P3): GPIO and UART Development Report

Oct 12, 2026 · @Roktim

Overview

P3 owns the hardware drivers of MiniOS, and two of the three, GPIO and UART, are built and verified on a Nucleo-F401RE board. ADC is the next driver.

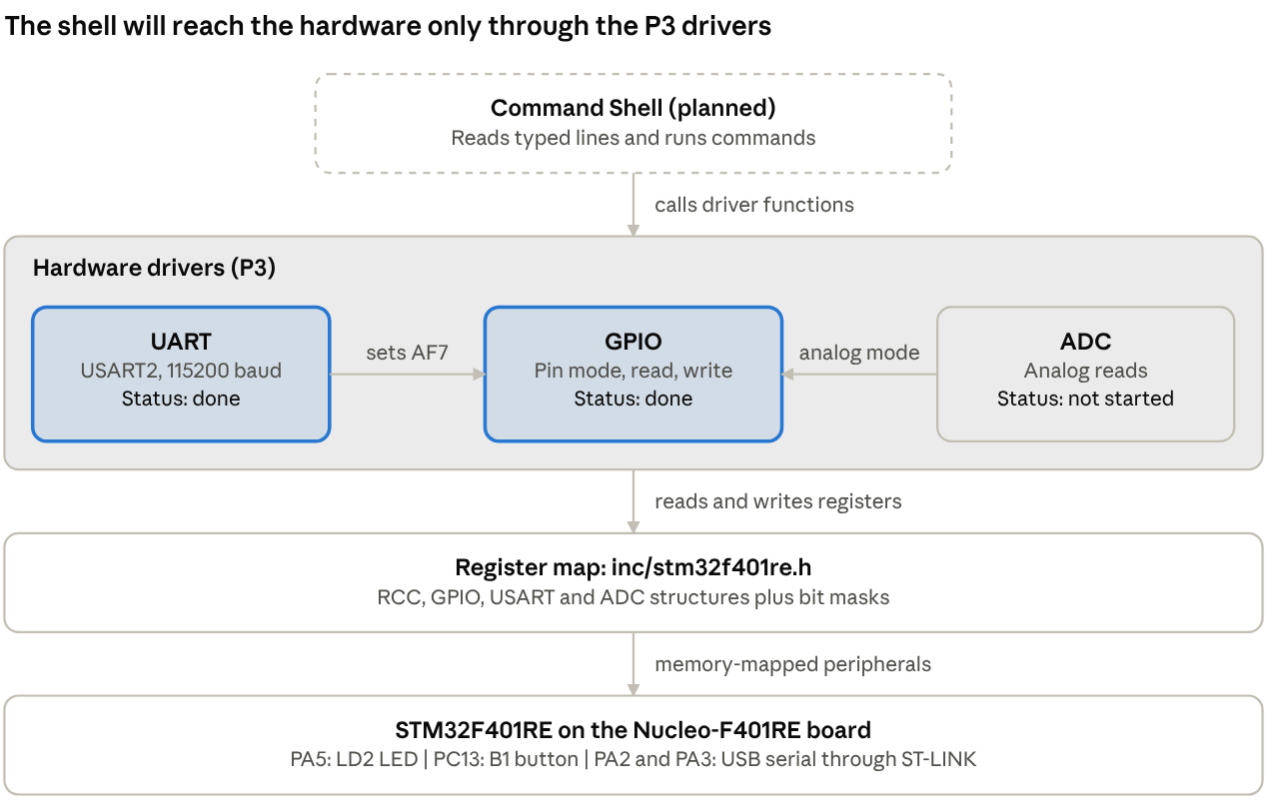
MiniOS is a minimal bare-metal operating system for the STM32F401RE (ARM Cortex-M4). Drivers are the layer that lets the rest of the system talk to the chip's pins and peripherals without touching registers directly. This report explains what each driver does, how it was built, and how any team member can test every function with one board.

Driver	What it does	Status	Proof on the board
GPIO	Configures pins and drives or reads them	Done	LED blink on PA5, button read on PC13
UART (USART2)	Sends and receives text over USB serial at 115200 baud	Done	Text printed, keystrokes echoed, line editing works
ADC	Reads analog voltages as numbers	Not started	None yet

The code is on branch `p3-hardware-drivers` of the team repository `emOS` and is not merged into `main` yet.

Where the drivers fit in MiniOS

The P3 drivers sit between the planned command shell and the chip, and both UART and ADC rely on GPIO to set up their pins.



layer diagram · shell, drivers, register map, hardware

The shell will call UART functions for typed input and output and GPIO functions for pins. Each driver reaches the chip only through the shared register map.

Hardware, toolchain and conventions

Everything runs on one Nucleo-F401RE board, is built with the Arm GCC cross-compiler and is flashed through the board's built-in ST-LINK.

Item	Detail
Board	Nucleo-F401RE with an STM32F401RE chip (ARM Cortex-M4)
On-board parts used	LD2 green LED on PA5; B1 blue user button on PC13; ST-LINK USB port with a virtual serial port wired to USART2 (PA2 TX, PA3 RX)
Clock	Default 16 MHz internal oscillator (HSI), no PLL; USART2's clock is also 16 MHz
Compiler	`arm-none-eabi-gcc` with `-std=c99 -Os -mthumb -mcpu=cortex-m4` and hardware float

Item	Detail
Warnings	`-Wall -Wextra -Wshadow -Wmissing-prototypes -Wstrict-prototypes -Wundef` and more, so sloppy code fails to build
Flash tool	`st-flash` from the stlink package, run by `make flash`, writing at address `0x8000000`
Serial terminal	`picocom` at 115200 baud, 8 data bits, no parity, 1 stop bit
Libraries	None. No HAL and no CMSIS: registers are accessed directly

The files involved:

File	Role
`inc/stm32f401re.h`	Register map: RCC, GPIO, USART and ADC structures plus the bit masks used
`inc/gpio.h`, `src/gpio.c`	GPIO driver
`inc/uart.h`, `src/uart.c`	UART driver
`inc/adc.h`, `src/adc.c`	ADC driver, skeleton only so far
`src/main.c`	Test program, swapped for each test below

Conventions the drivers follow, matching the existing `systick` module:

- Headers go in `inc/` and sources in `src/`, in one flat layout.
- Function names follow `module_verb`, such as `gpio_init`, `uart_putc` and `systick_init`.
- Headers only declare functions. All register definitions live in the shared `stm32f401re.h`.
- Every new `.c` file must be added to the `OBJS` list in the Makefile, because it does not detect files by itself.

Build, flash and serial terminal

Every test in this report uses the same loop: put a test program in `src/main.c`, build it, flash it, then watch the LED or the serial terminal.

One-time setup on macOS (check each tool with `arm-none-eabi-gcc --version`, `make --version`, `st-info --probe` and `picocom --version`):

```
brew install --cask gcc-arm-embedded
brew install stlink picocom
```

The loop:

1. Get the code: `git checkout p3-hardware-drivers`, then `git pull`. After a merge, use `main` instead.
2. Replace `src/main.c` with the test program from the section you are running.
3. Build with `make clean` and then `make`. A successful build prints nothing.
4. Plug in the board and flash with `make flash`. Success ends with `Flash written and verified! jolly good!`.
5. For UART tests, find the port with `ls /dev/tty.usb*` and open it with `picocom -b 115200 /dev/tty.usbmodemXXXX`.

Useful details:

- Exit picocom with Ctrl+A, then Ctrl+X.
- The black RESET button restarts the program. Use it when a test prints a message only once at startup, so you can open the terminal first and then catch it.
- The port number can change after unplugging or reflashing, so run `ls /dev/tty.usb*` again each time. On Linux the port usually appears as `/dev/ttyACM0`, and on Windows as a COM port in Device Manager.
- Close every other serial program before opening picocom. Two programs on one port can split the incoming data.
- Run `make clean` after switching branches or machines. Stale `.d` files can cause a `No rule to make target ... stdint.h` error.

GPIO: what it is and why MiniOS needs it

GPIO (general-purpose input/output) is how the chip reads and drives its physical pins, and the UART and ADC drivers both depend on it.

The STM32F401RE groups its pins into ports named GPIOA, GPIOB, GPIOC, GPIOD, GPIOE and GPIOH, each with up to 16 pins. Every pin is set to one of four modes before use:

Mode	What the pin does	Used in MiniOS for
Input	Reads whether the pin voltage is high or low	B1 user button on PC13

Mode	What the pin does	Used in MiniOS for
Output	Drives the pin high (3.3 V) or low (0 V)	LD2 status LED on PA5
Alternate function	Hands the pin to a peripheral such as USART2	PA2 and PA3 for UART
Analog	Leaves the pin free for the ADC to measure	ADC input pins, planned

How it works at a glance: each pin owns a few bits in several configuration registers. The driver sets those bits to choose the mode, output type, speed and pull resistor. After that, writing the output register changes the voltage on the pin, and reading the input register reports the voltage that is present.

Why it matters for MiniOS: the LED gives the kernel a visible sign of life, the button gives it a simple input, and the alternate-function mode is what connects the UART's transmit and receive lines to the outside world. Without a working GPIO driver the serial shell cannot even start.

GPIO: design and implementation

The GPIO driver is six small functions on top of one shared register map, written so that each register change is simple and cannot disturb neighbouring pins.

The register map in `stm32f401re.h` describes each port as a `GPIO_TypeDef` structure whose fields follow the hardware layout. Macros such as `GPIOA` and `GPIOC` point that structure at the port's base address. The ports sit `0x400` bytes apart (GPIOA at `0x40020000`, GPIOB at `0x40020400`, and so on, with GPIOH at `0x40021C00`).

Registers the driver uses:

Register	Bits per pin	What it controls
`MODER`	2	Mode: `00` input, `01` output, `10` alternate function, `11` analog
`OTYPER`	1	Output type: push-pull or open-drain
`OSPEEDR`	2	Output speed: low, medium, fast or high
`PUPDR`	2	Internal pull-up, pull-down or none
`AFR[0]`, `AFR[1]`	4	Alternate function number 0 to 15 (`AFR[0]` for pins 0 to 7, `AFR[1]` for pins 8 to 15)

Register	Bits per pin	What it controls
`IDR`	1	Input data: the level currently on the pin
`ODR`	1	Output data
`BSRR`	1 set bit and 1 reset bit	Atomic set (bit `n`) or reset (bit `n+16`) of pin `n`
`RCC->AHB1ENR`	1 per port	Clock enable. A port's registers do nothing until its bit is set

The functions:

Function	What it does
`gpio_clock_enable(port)`	Sets the port's bit in `RCC->AHB1ENR`. The bit number is `(port address - GPIOA address) / 0x400`
`gpio_init(port, pin, config)`	Enables the clock, writes the 2-bit `MODER` field, writes `OTYPER` and `OSPEEDR` for output and alternate-function modes, always writes `PUPDR`, and calls `gpio_set_af` for alternate-function mode
`gpio_set_af(port, pin, af)`	Picks `AFR[0]` or `AFR[1]` from the pin number, clears that pin's 4-bit field, then writes `af`
`gpio_write(port, pin, state)`	Writes `BSRR` bit `pin` to set the pin or bit `pin+16` to clear it
`gpio_toggle(port, pin)`	Flips the pin with `ODR ^= (1 << pin)`
`gpio_read(port, pin)`	Returns the pin's bit from `IDR` as `GPIO_PIN_HIGH` or `GPIO_PIN_LOW`

A typical call configures the LED pin like this:

```
gpio_config_t led = {
    .mode  = GPIO_MODE_OUTPUT,
    .otype = GPIO_OTYPE_PUSH_PULL,
    .speed = GPIO_SPEED_LOW,
    .pull  = GPIO_PULL_NONE,
    .af    = 0
};
gpio_init(GPIOA, 5, &led);
```

Design choices worth mentioning to a reviewer:

- **Config struct.** `gpio_config_t` carries mode, type, speed, pull and alternate function, so `gpio_init` has three arguments instead of seven.
- **Clock bit from the address.** One line of arithmetic works for every port, with no if/else chain to extend.
- **Clear, then set.** Multi-bit fields are cleared with a mask before the new value is written, so old bits never linger.
- **Only the relevant registers.** `OTYPER` and `OSPEEDR` are written only for output and alternate-function pins, because they have no effect on inputs.
- **`BSRR` for writes.** A single write sets or clears one pin without reading the register first, so it cannot overwrite other pins even if an interrupt touches the same port.
- **`gpio_toggle` caveat.** It uses a read-modify-write on `ODR`, which is fine while nothing else changes that port from an interrupt.

GPIO: testing each function

Three short tests cover the GPIO functions one at a time, and each one gives a result you can see on the board without a serial terminal.

All three use the same program. Copy it into `src/main.c`, put the test body inside the `while` loop, then build and flash as described earlier.

```
#include <stdint.h>
#include "systick.h"
#include "gpio.h"

int main(void)
{
    gpio_config_t led = {
        .mode = GPIO_MODE_OUTPUT, .otype = GPIO_OTYPE_PUSH_PULL,
        .speed = GPIO_SPEED_LOW, .pull = GPIO_PULL_NONE, .af = 0
    };
    gpio_config_t button = {
        .mode = GPIO_MODE_INPUT, .otype = GPIO_OTYPE_PUSH_PULL,
        .speed = GPIO_SPEED_LOW, .pull = GPIO_PULL_NONE, .af = 0
    };

    gpio_init(GPIOA, 5, &led);      /* LD2 on PA5 */
    gpio_init(GPIOC, 13, &button);  /* B1 on PC13 */
    systick_init(16000 - 1);        /* 1 ms tick, needed by delay_ms */
```

```
    while (1) {
        /* test body goes here */
    }
}
```

Test G1: gpio_write

Test body:

```
gpio_write(GPIOA, 5, GPIO_PIN_HIGH);
delay_ms(1000);
gpio_write(GPIOA, 5, GPIO_PIN_LOW);
delay_ms(1000);
```

- **Expected:** LD2 stays on for 1 second, then off for 1 second, repeating.
- **What it proves:** `gpio_write` sets the pin through the `BSRR` set bit and clears it through the reset bit. It also proves `gpio_init` configured PA5 as a working output.
- **Status:** each state has been seen on our board in the combined prompt test (`led on` and `led off`). This exact loop is ready to run.

Test G2: gpio_toggle

Test body:

```
gpio_toggle(GPIOA, 5);
delay_ms(500);
```

- **Expected:** LD2 blinks steadily, on for half a second and off for half a second.
- **What it proves:** `gpio_toggle` flips the pin through `ODR`, and the port A clock enable inside `gpio_init` works. If the clock bit were wrong, the LED would never change.
- **Status:** passed. This was the first test to run through the driver on the board.

Test G3: gpio_read

Test body:

```
if (gpio_read(GPIOC, 13) == GPIO_PIN_LOW) {
    gpio_write(GPIOA, 5, GPIO_PIN_HIGH);
} else {
    gpio_write(GPIOA, 5, GPIO_PIN_LOW);
}
```

- **Expected:** LD2 is off at rest, turns on while you hold the blue B1 button, and turns off again when you release it.

- **What it proves:** `gpio_read` returns the true level of an input pin. B1 is active-low (high when released, low when pressed), so the test checks for `GPIO_PIN_LOW`. It also proves the port C clock enable, because the read only works if port C is clocked.
- **Status:** passed, after correcting our first assumption about the button's polarity.

Functions covered indirectly

- **`gpio_clock_enable`** is called inside `gpio_init`. Tests G2 and G3 prove it for ports A and C. Ports B, D, E and H use the same arithmetic but have not been run on hardware.
- **`gpio_set_af`** can only be observed through a peripheral, so it is proven by the UART transmit test later in this report, where text appears only if pins PA2 and PA3 are set to alternate function 7 correctly.

UART: what it is and why MiniOS needs it

UART (universal asynchronous receiver-transmitter) sends and receives bytes one bit at a time over two wires, and it is MiniOS's only way to talk to a person at a keyboard.

One wire carries data out of the chip (TX) and the other carries data in (RX). There is no shared clock, so both sides agree on a speed, the baud rate, in advance. MiniOS uses the common `8N1` frame: one start bit, 8 data bits, no parity and one stop bit. That is 10 bits per byte, so at 115200 baud the link moves about 11,520 bytes per second.

On the Nucleo board, USART2's pins PA2 (TX) and PA3 (RX) are wired to the ST-LINK chip, which presents them to the laptop as a USB serial port. A terminal program such as `picocom` therefore works as the MiniOS console with no extra hardware.

Why it matters for MiniOS:

- The command shell reads typed lines and prints results. The UART driver supplies both halves through `uart_read_line` and `uart_puts`.
- A bare-metal system has no screen, so UART text is the first practical way to see what the kernel is doing.
- It depends on the GPIO driver, which must switch PA2 and PA3 to alternate function mode before any byte can leave the chip.

UART: design and implementation

The UART driver sets up one USART peripheral for 8N1 at a chosen baud rate, then moves bytes by polling its status flags.

Registers the driver uses:

Register	Bits used	What it controls
`RCC->APB1ENR`	bit 17 (`USART2EN`)	Clock for USART2. USART1 and USART6 are clocked from `APB2ENR` instead
`BRR`	all 16 bits	Baud-rate divisor
`CR1`	`UE` (13), `M` (12), `TE` (3), `RE` (2)	Peripheral enable, word length (0 = 8 bits), transmitter enable, receiver enable
`SR`	`TXE` (7), `TC` (6), `RXNE` (5)	Status: transmit register empty, transmission complete, received byte waiting
`DR`	8 bits	Writing sends a byte. Reading returns the received byte and clears `RXNE`

Baud-rate math. With the default 16x oversampling, the whole 16-bit `BRR` value equals the peripheral clock divided by the baud rate. The reference manual splits it into a mantissa (bits 15 to 4) and a fraction (bits 3 to 0), but those are just two views of the same integer, so no manual split is needed. The driver adds half the baud rate before dividing, which rounds to the nearest value instead of truncating:

```
\mathrm{BRR} = \left\lfloor \frac{f_{CK} + \mathrm{baud}/2}{\mathrm{baud}}
\right\rfloor
```

For 16 MHz and 115200 baud: 16,000,000 / 115,200 = 138.89, which rounds to 139 (`0x8B` , mantissa 8 and fraction 11). The real baud rate is then 16,000,000 / 139 = 115,108, an error of about 0.08%, well inside what a serial link tolerates.

Initialisation order inside `uart_init` : enable the USART clock, write `BRR` , clear `M` and set `TE` and `RE` , and set `UE` last so every setting is in place before the peripheral switches on.

The functions:

Function	What it does
`uart_init(usart, baudrate, pclk_hz)`	Runs the initialisation sequence above for the given USART
`uart_putc(usart, c)`	Waits until `TXE` is set, then writes the byte to `DR`
`uart_puts(usart, s)`	Sends characters until the string's null terminator

Function	What it does
`uart_write(usart, data, len)`	Sends `len` raw bytes, with no terminator needed
`uart_getc(usart)`	Waits until `RXNE` is set, then returns the byte from `DR`
`uart_data_available(usart)`	Returns 1 if a byte is waiting, otherwise 0, without blocking
`uart_read_line(usart, buf, maxlen)`	Reads a line into `buf` and returns its length (details below)

Before `uart_init`, the two pins must be switched to alternate function 7 with the GPIO driver:

```
gpio_config_t usart2_pin = {
    .mode = GPIO_MODE_AF, .otype = GPIO_OTYPE_PUSH_PULL,
    .speed = GPIO_SPEED_FAST, .pull = GPIO_PULL_NONE, .af = 7
};
gpio_init(GPIOA, 2, &usart2_pin);   /* TX */
gpio_init(GPIOA, 3, &usart2_pin);   /* RX */
uart_init(USART2, 115200, 16000000);
```

`uart_read_line` behaviour, written for the future shell:

- Enter (carriage return or line feed) ends the line and is not stored.
- Each typed character is echoed back by the board, because the terminal's own local echo is off.
- Backspace (`0x08` or `0x7F`) removes the last stored character and erases it on screen by sending backspace, space, backspace. On an empty line it is ignored.
- It stops by itself after `maxlen - 1` characters, so a 64-byte buffer holds at most 63, and it always ends the text with a null byte.

Design choices worth mentioning to a reviewer:

- **Polling, not interrupts.** This matches the core milestone. Interrupt-driven receive with a ring buffer is a stretch goal, and `uart_data_available` is already in place so callers will not need to change.
- **One driver for three USARTs.** The peripheral is a parameter and the clock bit is chosen by comparing it with `USART1`, `USART2` and `USART6`. Only USART2 has been tested on hardware.
- **Pins stay in the GPIO driver.** `uart_init` does not touch pins, so the team can choose a different pin pair without editing the UART code.

UART: testing each function

Five tests cover the UART functions one at a time, and each result appears as text in the serial terminal.

All five use the same program, which touches GPIO only to switch PA2 and PA3 to alternate function 7 (UART cannot work without that). Put the test's setup before the loop and its body inside it, then build and flash. Open `picocom -b 115200 /dev/tty.usbmodemXXXX` and, when a test prints only at startup, press the black RESET button with the terminal open.

```
#include <stdint.h>
#include "systick.h"
#include "gpio.h"
#include "uart.h"

int main(void)
{
    gpio_config_t usart2_pin = {
        .mode = GPIO_MODE_AF, .otype = GPIO_OTYPE_PUSH_PULL,
        .speed = GPIO_SPEED_FAST, .pull = GPIO_PULL_NONE, .af = 7
    };

    gpio_init(GPIOA, 2, &usart2_pin);   /* TX */
    gpio_init(GPIOA, 3, &usart2_pin);   /* RX */
    uart_init(USART2, 115200, 16000000);
    systick_init(16000 - 1);            /* needed by delay_ms */

    /* test setup goes here (runs once) */

    while (1) {
        /* test body goes here */
    }
}
```

Test U1: uart_init, uart_putc and uart_puts

Test body:

```
uart_puts(USART2, "Hello from MiniOS UART!\r\n");
delay_ms(500);
```

- **Expected:** `Hello from MiniOS UART!` appears twice a second, as clean readable text.

- **What it proves:** `uart_init` set the clock, baud rate and control register correctly. Garbled characters would mean a wrong baud rate, so readable text confirms the `BRR` calculation. It also proves `uart_puts` and `uart_putc` with the `TXE` handshake, and `gpio_set_af`, since nothing leaves PA2 unless it is set to alternate function 7.
- **Status:** passed.

Test U2: uart_write

Test setup, before the loop:

```
const uint8_t msg[] = "MiniOS-IGNORED";
```

Test body:

```
uart_write(USART2, msg, 6);
uart_puts(USART2, "\r\n");
delay_ms(500);
```

- **Expected:** the word `MiniOS` appears twice a second, with no `-IGNORED` after it.
- **What it proves:** `uart_write` sends exactly the number of bytes asked for, without depending on a null terminator.
- **Status:** not run on the board yet. Run it before the demo.

Test U3: uart_getc

Test body:

```
char c = uart_getc(USART2);
uart_puts(USART2, "Got: ");
uart_putc(USART2, c);
uart_puts(USART2, "\r\n");
```

- **Expected:** nothing prints until you press a key. Each key then prints `Got:` followed by that key.
- **What it proves:** `uart_getc` waits for `RXNE`, returns the exact byte received, and the receive pin and the `RE` bit are working.
- **Status:** this stand-alone loop is ready to run. The function itself is proven through tests U4 and U5, which both call it.

Test U4: uart_data_available

Test body:

```
uart_puts(USART2, "Heartbeat: still alive\r\n");
if (uart_data_available(USART2)) {
    char c = uart_getc(USART2);
    uart_puts(USART2, "Echo: ");
    uart_putc(USART2, c);
    uart_puts(USART2, "\r\n");
}
delay_ms(500);
```

- **Expected:** `Heartbeat: still alive` appears twice a second without ever pausing. When you press a key, a line such as `Echo: x` appears within half a second.
- **What it proves:** `uart_data_available` does not block, so the loop keeps running while it waits for input. It also proves that sending still works after receiving.
- **Status:** passed. Our run also toggled LD2 each cycle, and the keys `x`, `y` and `r` were each echoed correctly.

Test U5: uart_read_line

Test setup, before the loop:

```
char line[64];
```

Test body:

```
uart_puts(USART2, "\r\n> ");
uart_read_line(USART2, line, sizeof(line));
uart_puts(USART2, "\r\nYou typed: ");
uart_puts(USART2, line);
uart_puts(USART2, "\r\n");
```

Try each input and compare:

Type this	Expected result	What it proves
`hello`, then Enter	`You typed: hello`	Basic line read, echo of each character, end at Enter
Enter alone	`You typed:` with nothing after it	An empty line is handled and still null-terminated
`hello`, Backspace twice, `p!`, Enter	`You typed: help!`	Backspace removes the last character from the buffer and from the screen

Type this	Expected result	What it proves
`hello world`, then Enter	`You typed: hello world`	A space is an ordinary character, not a line end
A sentence longer than 63 characters	The line is returned after 63 characters without waiting for Enter	The buffer limit holds and nothing is written past the end of the array

- **Status:** passed. All five inputs were run on our board.

Combined check: commands over the serial prompt

One program ties both drivers together: a command typed in the serial terminal changes the LED or reports the button, which previews how the MiniOS command shell will use these drivers.

The program is the combined `src/main.c` from the final stage of the UART work. It sets up the LED, the button and USART2, prints a banner, then loops: print a prompt, read a line with `uart_read_line`, compare it with the known commands and act on it. Flash it as it is and open the terminal. Press RESET with the terminal open to see the banner.

Command	Action	Reply printed
`led on`	Drives LD2 high with `gpio_write`	`LED turned ON`
`led off`	Drives LD2 low with `gpio_write`	`LED turned OFF`
`led toggle`	Flips LD2 with `gpio_toggle`	`LED toggled`
`button`	Reads B1 with `gpio_read`	`Button is PRESSED` or `Button is NOT pressed`
Anything else	Nothing	`Unknown command. Try: led on / led off / led toggle / button`
Enter alone	Nothing	A fresh prompt

A typical session:

```
--- MiniOS P3 Driver Integration Test ---
Commands: led on | led off | led toggle | button

> led on
LED turned ON

> button
Button is NOT pressed

> xyz
Unknown command. Try: led on / led off / led toggle / button
```

To test the pressed state, hold B1 and type `button` while still holding it. The reply should change to `Button is PRESSED`.

- **What it proves:** the drivers work together. Text received through UART is turned into a GPIO action, and a GPIO input is reported back through UART. That is the same path the command shell will use.
- **Status:** passed. Every command and the unknown-command reply were run on our board.
- **Why the LED does not blink on its own:** `uart_read_line` waits for typing, so nothing else runs in the loop. The LED changes only when a command says so. Blinking while listening would need interrupts, which are outside the core milestone.

Results summary

12 of the 13 GPIO and UART functions have passed on the board, and `uart_write` is the one still to run.

Function	Test	Result on our board
`gpio_clock_enable`	G2, G3	Passed for ports A and C
`gpio_init`	G1 to G3	Passed
`gpio_set_af`	U1	Passed
`gpio_write`	G1	Passed, both states seen
`gpio_toggle`	G2	Passed
`gpio_read`	G3	Passed
`uart_init`	U1	Passed, USART2 at 115200 baud

Function	Test	Result on our board
`uart_putc`	U1	Passed
`uart_puts`	U1	Passed
`uart_write`	U2	Not run yet
`uart_getc`	U3 to U5	Passed, proven through U4 and U5
`uart_data_available`	U4	Passed
`uart_read_line`	U5	Passed, all five inputs

The combined command-prompt check also passed. The ADC driver has no tests yet because it is not implemented.

Problems met and how each was fixed

Seven problems came up during development, and the first four are repository and build issues that any teammate could hit.

Problem	Cause	Fix
Driver files did not match the repo	The first skeleton assumed a `drivers/` folder and included `../inc/stm32f401re.h`, but the repo uses flat `src/` and `inc/`	Moved the files, changed the includes to `"stm32f401re.h"`, and added the new `.o` files to `OBJS` in the Makefile
`No rule to make target ... stdint.h` during `make`	Old `.d` dependency files committed from another machine recorded that machine's toolchain paths	Run `make clean`, then `make`
Build files showed up in `git status`	`.gitignore` only excluded `/build`, but output lands in `src/` and the repo root. A later fix typed every pattern on one line, which Git reads as a single pattern	One pattern per line (`*.o`, `*.d`, `*.elf`, `*.bin`, `*.hex`, `*.map`, `.DS_Store`), and `git rm --cached` on files already tracked
`No targets specified and no makefile found`	`make` was run from the parent folder	Run it from inside the repo folder

Problem	Cause	Fix
The button seemed to work backwards	B1 has an external pull-up: it reads high when released and low when pressed. The driver was correct and the test assumed the opposite	Test for `GPIO_PIN_LOW` to detect a press
The startup message never appeared in the terminal	The message was sent once at boot, before the terminal was open	Use messages that repeat, or press RESET with the terminal already open
Keys were received but no echo showed	LD2 toggled on every key, so receive worked, but the echo did not appear in the terminal	Restarted from the proven transmit build and retested with a repeating heartbeat plus a non-blocking receive check, and the echo then worked. The exact cause of the first failure was not isolated

Known limitations and next steps

The drivers are deliberately simple and polling-based, so their limits are mostly what they do not handle yet, and the next steps are the ADC driver and a Pull Request.

Limitations:

- **Blocking calls.** `uart_putc`, `uart_getc` and `uart_read_line` wait inside a loop, so nothing else runs meanwhile. Interrupt-driven receive with a ring buffer is a stretch goal in the proposal.
- **Only USART2 is tested.** The code can clock USART1 and USART6, but neither has run on hardware.
- **Only GPIO ports A and C are tested.** The other ports use the same arithmetic but have not been run.
- **Clock assumption.** The tests assume the default 16 MHz oscillator. If the clock tree changes, the `pclk_hz` value passed to `uart_init` and the SysTick reload value must change with it.
- **No receive error handling.** The driver ignores overrun and framing errors, and the data register holds one byte, so fast pasted text can lose characters.
- **`uart_write` is untested** on hardware (test U2).
- **ADC is not implemented.** `adc.c` and `adc.h` are skeletons.

Next steps:

1. Run test U2 on the board to close the last gap in the results table.
2. Implement the ADC driver (ADC1, single conversion, polling) and test it against a known voltage, printing readings over UART.
3. Open a Pull Request from `p3-hardware-drivers` into `main` for P1 to review. The `.gitignore` fix travels with it.
4. Agree with P1 where test programs live, because each test above replaces `src/main.c`.